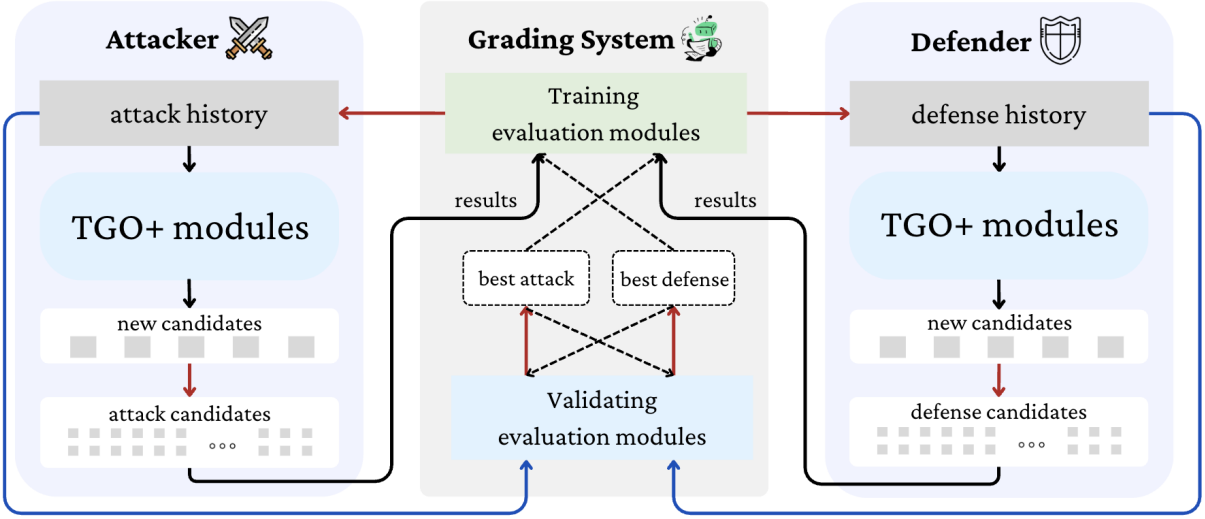

Figure 2: **Overview of the Co-evolutionary Adversarial Framework**. The system continuously co-optimizes attack and defense prompt candidates through interaction with a main application. Prompt candidates are evaluated based on the formula (1) and (2), and gradient-like feedback is used to iteratively evolve both attackers and defenders, encouraging robustness and adaptivity across adversarial interactions.

output three important values based on the original input and the attack candidate:

- S_{benign}: The score assigned to the original (benign) input without any adversarial prompts.
- S_{attacked}: The score assigned after injecting adversarial prompts into the original input.
- ASR: The **attack success rate**, which quantifies the probability that an attack bypasses detection by the defense.

After generating these values, we first calculate the relative score change ΔS_{rel} in Equation (1):

$$\Delta S_{\text{rel}} = \begin{cases} \frac{S_{\text{attacked}} - S_{\text{benign}}}{S_{\text{max}} - S_{\text{benign}}} & \text{if } S_{\text{benign}} < S_{\text{max}} \\ 0 & \text{if } S_{\text{benign}} \geq S_{\text{max}} \end{cases} \quad (1)$$

where S_{benign} means the maximum possible score defined by the grading system.

If $S_{\text{benign}} < S_{\text{max}}$, the relative score change is computed as the ratio between the observed improvement due to the attack and the maximum improvement that could possibly be achieved. Conversely, if $S_{\text{benign}} \geq S_{\text{max}}$, no further improvement is feasible, and ΔS_{rel} is set to zero. This normalization is critical because it accounts for the diminishing significance of score increments near the upper end of the grading scale. For instance, an increase from 8 to 9 carries greater weight than an increase from 1 to 2, as improvements become progressively harder to obtain as scores approach the maximum.

After computing the relative score change ΔS_{rel}, we rank and maintain the top-k adversarial attacks using the **attack score** defined in Equation (2).

$$S_{\text{attack}} = w_{\text{asr}} \cdot (\text{ASR})^{p_{\text{asr}}} + w_{\text{sc}} \cdot (\Delta S_{\text{rel}})^{p_{\text{sc}}} \quad (2)$$

where w_{asr} and w_{sc} are weights for the ASR and ΔS_{rel}, respectively, and p_{asr} and p_{sc} are power parameters to control the sensitivity of each term.

The weighting coefficients (w) determine the relative importance of the two metrics in the overall score—for example, assigning a larger w_{asr} prioritizes attacks that are more difficult for the defense to detect. The power parameters (p) modulate how strongly changes at different regions of the metric's range influence the score—for instance, a larger p_{asr} amplifies the effect of improvements in high-success regions (e.g., from 0.8 to 0.9) relative to low-success regions (e.g., from 0.1 to 0.2).

After the training procedure, the best-performing attack (ATK^i_{best}) from the top-k pool is chosen with the highest attack score on the validation set using $Val()$ in the grading system. Then, the selected adversarial prompts will be fixed for the next defender evolution cycle, ensuring that the defender is trained against the most challenging known threat at the time.

By leveraging LLM-based editing, gradient-guided refinement, and evaluation signals from the grading system, the attacker adaptively explores the adversarial prompt space, driving the co-evolutionary process forward.

3.1.2 Defender

The defender module aims to develop prompts that are robust against adversarial attacks and capable of eliciting accurate system responses. Unlike the attacker, which seeks to exploit model weaknesses, the defender focuses on maintaining reliability under adversarial pressure.

Following the co-evolutionary setup, the defender evolves in response to a fixed attacker. After each attacker cycle, the best-performing attack prompt is used as the evaluation context against which the defender is trained. The evolution process mirrors that of the attacker: new defense candidates ($DEF_Cand_j^i$) are generated via the TGO^+ module, and a top-k pool of defense prompts denoted as DEF_j^i is maintained via $Eval()$.

While the underlying mechanics—generation, evaluation, and selection—are symmetric to the attacker's process, the defender faces a different optimization goal. Rather than maximizing disruption, the defender is trained to neutralize the attack while preserving the semantic intent and correctness of responses. This often requires precise prompt calibration and semantic grounding, especially in high-stakes settings.

Ultimately, the defender provides a moving target for the attacker, contributing to the dynamic equilibrium of the co-evolutionary training process.

To evaluate the effectiveness of the defense prompt, we ask the LLM in the grading system to output two important values: **True Positive Rate (TPR)**, which denotes the probability that the defense correctly detects an attack, and **True Negative Rate (TNR)**, which denotes the probability that the defense correctly identifies a benign input as non-attacked.

After generating these two values, we will calculate the **defense score** in Equation (3):

$$S_{defense} = w_{tp} \cdot (TPR)^{p_{tp}} + w_{tn} \cdot (TNR)^{p_{tn}} \quad (3)$$

where w_{tp} and w_{tn} are weights for the True Positive Rate (TPR) and True Negative Rate (TNR), respectively, and p_{tp} and p_{tn} are their corresponding power parameters. Similar to the attack score, the coefficients and power parameters in the defense score allow for a nuanced and flexible evaluation of the effectiveness of the defense prompt.

Similarily, the best defense (DEF_{best}^i) is chosen from the top-k pool with the highest defense score on the validation set using $Val()$ in the grading system after the training process.

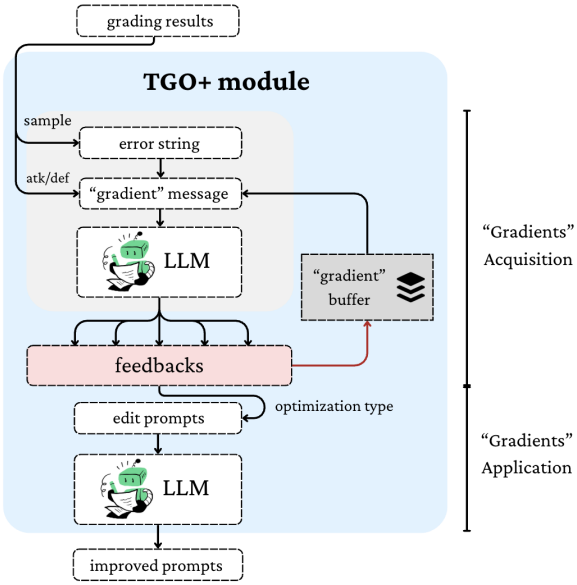

Figure 3: **Overview of the Textual Gradient Optimization (TGO) module.** The TGO module iteratively improves prompts by simulating gradient-based optimization using language model feedback. Grading results are sampled to construct error strings and generate gradient messages, which are then processed by an LLM to obtain feedback. These feedbacks are used to guide the editing of prompts based on the optimization type (e.g., attack or defense).

3.2 TGO+ for Prompt Optimization

Inspired by the TGO framework proposed in Pryzant et al. (2023), we adopt a modular design to optimize prompts through gradient-like updates in natural language space. Each **TGO+ module** is dedicated to a specific optimization goal (e.g., attack or defense) and operates in two stages: *gradient acquisition* and *gradient application*, as illustrated in Figure 3. The overall algorithm for TGO+ can be seen in Algorithm 2.

3.2.1 Gradient Acquisition

For each prompt and its gradient results (ASR, ΔS_{rel} for attack prompt; TPR, TNR for defense prompt), the module first collects the errors on these gradient results. These errors are then combined with a task-specific instruction and prompted to LLM, and LLM returns several feedback messages serving as the textual gradients—i.e., suggestions indicating how the prompt could be improved.

To ensure diverse learning, recent feedback messages are stored in a gradient buffer. This buffer encourages diversity in the optimization trajectory by prompting the LLM to generate alternative gradients even when the same input prompt is provided.

3.2.2 Gradient Application

In the second stage, the feedback is synthesized into a set of guidance messages based on the optimization type (e.g., ASR optimization for attack / TPR optimization for defense). These are used to construct an edit prompt that instructs the LLM to revise the original candidate prompt accordingly. After all these prompt candidates are generated, they are sent to $Eval()$ in the grading system to evaluate their effectiveness.

TGO+ enables gradient-like prompt updates without requiring access to model internals, making it compatible with black-box LLMs such as GPT-4o and Gemini-2.5-flash.

3.2.3 Key Innovations

Our implementation of TGO+ introduces several key innovations that differentiate it from the original work:

- **Multi-Route Gradient Optimization:** To enhance the optimization process, we employ a multi-route gradient strategy. This means that for each prompt, we generate textual gradients based on multiple optimization type. For instance, the Attackerś prompts are optimized based on either ASR or the relative score change. Similarly, the Defenderś prompts are optimized based on TPR or TNR. This allows for a more holistic and effective optimization process.

- **Gradient Buffer:** We introduce a gradient buffer that stores past textual gradients. This prevents the model from repeatedly making the same mistakes and encourages the exploration of novel optimization pathways.

4 Experimental Setup

4.1 Dataset

The dataset for our experiment can be separated into two parts: one for training, and one for real-world evaluation. During the training phase, we use a total of 50 GPT-generated benign articles. For these 50 articles, 40 of them are used during training, and 10 of them are used for validation. For the real-world evaluation phase, we use 143 malicious articles collected from student submissions in the previous course at National Taiwan University, which is the same course in Chiang et al. (2024). These articles contain a wide variety of successful prompt injections that achieve full scores without being detected by the defense, which serve as the baseline for calculating the True Positive Rate (TPR). Furthermore, we select another 100 benign articles from the course. These 100 articles do not contain any injections, which serve as the baseline for calculating the True Negative Rate (TNR) of the defense. All student-submitted articles (143 malicious ones + 100 benign ones) have been manually modi- fied to anonymize personal data and for copyright purposes, while preserving their original strategic intent

4.2 Experimental Procedure

To ensure the reliability and stability of our findings, all experiments were conducted three times. The results presented in this paper are the average values from these three runs. We also calculated the standard deviation of these experiments. This statistic calculation mitigates the impact of stochasticity in the training process and provides a more robust measure of performance.

4.3 Hyperparameters

The default hyperparameters used in our experiments are summarized in Appendix A.3. These settings were used for the baseline experiment, and variations are explored in the ablation studies.

5 Results

To comprehensively evaluate our framework, we benchmark its performance against several established baseline methods and analyze its iterative improvement over the training process.

5.1 Overall Evaluation

We evaluate the defense effectiveness of our framework against three baseline mechanisms, including Perplexity-based Detection (Alon and Kamfonas, 2023), LLaMA 3.1 Guard (Inan et al., 2023), and the "Human-Crafted Prompt" defense, which is proposed in Chiang et al. (2024) to defend against real-world attacks (See Appendix A.6). For all these methods, we evalaute the defense against the real-world articles (147 malicious articles + 100 benign articles). The results, summarized in Table 1, show that our method achieves state-of-the-art defense performance. We present our results at both an early stage (Iteration 4) and the final stage (Iteration 8) of the GAN training.

As shown, our defender at Iteration 4 already outperforms the strong LLaMA 3.1 Guard baseline. By Iteration 8, our method keeps improving,